HiAgent v1.6.0智能体接口文档

API 文档

调用示例（以对话类聊天为例）



一、调用说明

第一步：在智能体预览页右侧，获取API调用的URL和鉴权用的{{ ApiKey }}

第二步：

- 在请求header加上Apikey={{ ApiKey }}，body参数中包括AppKey，也赋值为{{ ApiKey }}；
- 所有接口需要在Body中加上UserID，UserID为调用API的用户ID（用户标识，用于定义终端用户的身份，方便检索、统计。 由开发者定义规则，需保证用户标识在应用内唯一）；

第三步：开始对话

- 选择一个合适的应用，获取应用AppID，调用CreateConversation生成一个会话ID；
- 调用ChatQuery接口（所有流式接口都是SSE），传入会话ID，即可开始聊天，返回流式的对话数据；
- 停止聊天：在流式输出的过程中调用StopMessage接口，即可停止聊天；
- 重新生成：如果需要重新生成聊天结果，调用QueryAgain，可重新生成聊天结果；
- 结果反馈：调用Feedback接口，传入消息ID，可对回答进行反馈评价（赞或踩）；
- 查看聊天历史：获取会话的聊天历史：GetConversationMessages；

服务：ChatService

API 列表概览

ChatQuery: 对话类聊天

ChatQueryV2: 对话类聊天V2

QueryAgain: 重新生成回复

GetAppConfigPreview: 获取应用配置

GetConversationList: 获取对话列表

GetConversationInputs: 获取对话变量输入

CreateConversation: 创建会话

UpdateConversation: 更新会话

DeleteConversation: 删除会话

StopMessage: 停止响应

GetConversationMessages: 获取会话历史消息列表

GetMessageInfo: 获取消息详情

DeleteMessage: 删除消息

Feedback: 回答反馈评价（赞或踩）

SetMessageAnswerUsed: 多组回答时，设置某个回答为默认使用的回答

GetSuggestedQuestions: 获取提问建议

RunAppWorkflow: 测试运行工作流(异步)

QueryRunAppProcess: 查询测试运行工作流运行进展(异步)

ListOauth2Token: 查询用户oauth2登录信息的接口

QueryAgainV2: 重新生成回复V2

ClearMessage: 清除消息

SyncRunAppWorkflow: 测试运行工作流(同步)

CallTriggerWebhook: 触发事件触发器

API 接口

创建会话

- 接口说明：对话类聊天接口，需要先生成会话，会话ID为AppConversationID，后续的聊天接口ChatQuery都需要传入会话ID。
- 请求header说明：

a. 请求header需要加上Apikey={{ ApiKey }};

- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）,已废弃可不传；
  b. Inputs：变量输入，可选，根据应用编排时设置的变量确定；
  c. UserID：用户标识，用于定义终端用户的身份，方便检索、统计。 由开发者定义规则，需保证用户标识在应用内唯一。（长度：1-20）；
- response body参数说明：
  a. Conversation：AppConversationID,后续的聊天接口ChatQuery都需要传入会话ID;ConversationName,会话默认名（可修改）；
  b. BaseResp：系统参数，不关注；

Post /create_conversation

请求 (app.CreateConversationRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
Inputs	map[string, string]	false	变量输入
UserID	string	true	用户ID

响应 (app.CreateConversationResponse):

参数	类型	是否必填	说明
Conversation	app.AppConversationBrief	false	对话信息
BaseResp	base.BaseResp	false	

	Postman示例	py代码示例
请求示例:	代码块 curl --location '{{apiurl}}/api/proxy/api/v1/create_conversation' \	代码块 def create_conversation(user_id: str) -> str: """ 创建会话 :param user_id: 用户id

```
--header 'Apikey: co6dd4i1hp0kieia137g' \
--header 'Content-Type: application/json' \
--data '{
    "Inputs": {
        "var": "variable"
    },
    "UserID": "321"
}'
```

```
    :return: 对话id
    :raises: ValueError, requests.exceptions.RequestException
    """
    url = base_url + "create_conversation"
    headers = {'Apikey': api_key, 'Content-Type': 'application/json'}
    data = {"UserID": user_id, "Inputs": {"var": "variable"}}

    try:
        response = requests.post(url, headers=headers, data=json.dumps(data))
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        raise requests.exceptions.RequestException(f"网络请求失败: {e}")

    try:
        result = response.json()
    except json.JSONDecodeError as e:
        raise ValueError(f"响应 JSON 解析失败: {e}")

    if "Conversation" not in result:
        raise ValueError("API 返回的结果缺少 'Conversation' 字段")

    if "AppConversationID" not in result["Conversation"]:
        raise ValueError("API 返回的结果缺少 'AppConversationID' 字段")

    return result["Conversation"]["AppConversationID"]
```

返回示例:

代码块

```
{

"Conversation": {
```

```
    "AppConversationID": "ct4m7gmoc93a9ns2sngg",
    "ConversationName": "新的会话",
    "CreateTime": "",
    "LastChatTime": "",
    "EmptyConversation": false
    },
    "BaseResp": null
}
```


对话类聊天（建议使用V2接口）

- 接口说明： 对话类聊天接口，需要先生成会话。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }};
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation接口生成；
  c. UserID：用户标识，用于定义终端用户的身份，方便检索、统计。 由开发者定义规则，需保证用户标识在应用内唯一。（长度：1-20）；
  d. Query：用户输入/提问内容；
  e. ResponseMode：streaming 流式模式（推荐）。基于 SSE（Server-Sent Events）实现类似打字机输出方式的流式返回。 blocking 阻塞模式，等待执行完毕后返回结果；

- response body参数说明：
  a. Data：SSE格式的输出；

Post /chat_query

请求 (app.ChatQueryRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧对话ID
Query	string	true	用户询问内容
ResponseMode	string	false	返回数据的格式，枚举值：streaming、blocking，默认：streaming
QueryExtends	app.QueryExtendsInfo	false	询问扩展信息
UserID	string	true	用户ID

响应 (app.MessageStreamResponse):

参数	类型	是否必填	说明
Data	string	false	EventStream(data: json)

	Postman示例	py代码示例	备注
请求示例：	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/chat_query' \ 2 --header 'Apikey: co6dd4i1h	代码块 1 def chat_query(user_id: str, app_conversation_id: str, content: str): 2 """ 3 对话类聊天 4 :param user_id: 用户id 5 :param app_conversation_id: 对话id 6 :param content: 用户询问的内容 7 :return:	Files信息包括obs链接，关于如何将文件上传obs参考文档：Hi Agent Up服务接口调用说明 流式请求时： 请求头注意 'Accept': 'text/event-stream'

```
p0kieia13
7g' \
--header
'Content-
Type:
applicati
on/json'
\
--data '{

"Query":
"给我讲个故
事",

"AppConve
rsationID
":
"co6deaa1
hp0kieia1
3bg",

"Response
Mode":
"streamin
g",

"UserID":
 "321",

"QueryExt
ends": {

"Files":
[

 {

"Name":
"图
片.jpeg",

"Path":
"upload/f
ull/59/e8
/0304b770
```

```
    """
    url = base_url +
"chat_query"
    headers = {'Apikey':
api_key,
               'Content-
Type': 'application/json',
               'Accept':
'text/event-stream'}
    data = {"UserID":
user_id,

"AppConversationID":
app_conversation_id,
            "Query":
content,

"ResponseMode":
"streaming"}
    try:
        response =
requests.post(url,
headers=headers,
data=json.dumps(data),
stream=True)
    except Exception as e:
        raise
ParamsException(403,
str(e))
    return response
```

代码请求时：request 设置 `stream=True`

```
fd1dfebe806bf1ade61a4bc5ded18f969a1c8f803998eeb60573",
14
"Size": 47525,
15
"Url": "{{apiurl}}/api/proxy/down?Action=Download&Version=2022-01-01&Path=upload%2Ffull%2F59%2Fe8%2F0304b770fd1dfebe806bf1ade61a4bc5ded18f969a1c8f803998eeb6
```

返回示例：

流式

代码块

```
1 {
2 event:text
3 data:data:{"event":
"message_start",
"task_id":
"6b2f4720-a14e-
```

代码块

```
1 if __name__ == '__main__':
2     try:
3         uid = "321"
4         conversation_id = create_conversation(uid)
5 print(f"conversation_id: {conversation_id}")
6
7         question = "给我讲个故事"
8         resp = chat_query(uid,
```

Data是标准的sse流式返回，用户需要自行解析data字段。字段含义如下：

- event：事件类型，枚举值
  - message_start：消息开始
  - message：消息
  - message_end：消息结束

```
413d-
9783-
024442a58
e7d"}
4
5   event:tex
    t
6   data:data
    :
    {"event":
    "message"
    ,
    "task_id"
    :
    "6b2f4720
    -a14e-
    413d-
    9783-
    024442a58
    e7d",
    "id":
    "01JDHDFN
    H1XMJG89S
    EP2F49HDV
    ",
    "answer":
     "你",
    "created_
    at":
    173253013
    4,
    "conversa
    tion_id":
    "01JDHDFN
    GYRB130PZ
    SEDE8R70B
    "}
7   }
```

```
conversation_id, question)
9           start_time =
    time.time()
10          task_id = None
11
12          for line in
    resp.iter_lines(chunk_size=
    4):
13              if line:  # 过滤
    掉空行
14                  line =
    line.decode("utf-8")
15                  if
    line.startswith("data:"):
16                      data =
    line.strip("data:").strip()
17                      if
    data:
18
    try:
19
     data = json.loads(data)
20
     if "event" in data and
    data["event"] == "message":
21
         print(data["answer"],
    end="")
22
     if "task_id" in data:
23
         task_id =
    data["task_id"]
24
    except
    json.JSONDecodeError as e:
25
     print(f"JSON 解析失败:
    {e}")
26
    except KeyError as e:
27
     print(f"返回数据缺少必要字
    段: {e}")
28
29          print("\n")
```

- message_output_start：开始输出
- message_output_end：输出结束
- message_cost：消息消耗
- suggestion：问题建议
- suggestion_cost：问题建议消耗
- agent_thought：思考
- agent_thought_end：思考结束
- message_replace：消息替换
- message_failed：消息失败
- knowledge_retrieve：知识库检索
- knowledge_retrieve_end：知识库检索结束
- agent_jump：agent跳转事件
- agent_take_over：agent接管最终回答的事件
- intention：意图
- qa_retrieve：问答库检索

```
            print(f"整体耗时: {time.time() - start_time}")

        except requests.exceptions.RequestException as e:
            print(f"网络请求失败: {e}")
        except ValueError as e:
            print(f"API 返回结果异常: {e}")
        except Exception as e:
            print(f"未知错误: {e}")
```

- qa_retrieve_end：问答库检索结束
- knowledge_graph_retrieve：图谱库检索
- knowledge_graph_retrieve_end：图谱库检索结束
- terminology_retrieve：术语库检索
- terminology_retrieve_end：术语库检索结束
- tool_message：消息节点
- think_message_output_start：思考开始
- think_message：思考内容
- think_message_output_end：思考结束

- task_id：任务ID，等效于当前对话的全局消息ID

			• id：消息ID，包括当前对话的全局消息ID和嵌套的子消息ID：全局消息ID作为其他接口的MessageID入参；对话时如果调用工具等，会生成子消息ID。流式场景下，最外层的message_start、message_end里的ID为全局消息ID；非流式场景只会返回全局消息ID。 • answer：模型回复内容； • created_at：消息创建时间； • conversation_id：模型会话ID，作用与app_conversation_id一致。区别为：app_conversation_id作为用户感知的一个会话的标识，具有业务属性；conversation_id只作为底层模型交互的会话的标识，屏蔽用户感知
返回示例：非流式	代码块 1 { 2 event:text 3 data:data:{"event": "message_	代码块 1 if __name__ == '__main__': 2 try: 3 uid = "321" 4 conversation_id = create_conversation(uid) 5 print(f"conversation_id: {conversation_id}")	chat_query接口非流式返回也是SSE结构的数据（历史遗留问题），该接口不再推荐，建议使用chat_query_v2

```
end",
"task_id"
:
"6b2f4720
-a14e-
413d-
9783-
024442a58
e7d",
"id":
"01JDHDFN
H1XMJG89S
EP2F49HDV
",
"answer":
 "你好，回
答完毕",
"created_
at":
173253013
4,
"conversa
tion_id":

"01JDHDFN
GYRB130PZ
SEDE8R70B
"}
}
```

```python

        question = "给我讲个故事"
        resp = chat_query(uid, conversation_id, question)
        start_time = time.time()
        task_id = None

        for line in resp.iter_lines(chunk_size=4):
            if line:  # 过滤掉空行
                line = line.decode("utf-8")
                if line.startswith("data:"):
                    data = line.strip("data:").strip()
                    if data:
                        try:
                            data = json.loads(data)
                            if "event" in data and data["event"] == "message_end":
                                print(data["answer"], end="")
                            if "task_id" in data:
                                task_id = data["task_id"]
                        except json.JSONDecodeError as e:
                            print(f"JSON 解析失败: {e}")
                    except KeyError as e:
```

```
            print(f"返回数据缺少必要字段: {e}")

            print("\n")
            print(f"整体耗时: {time.time() - start_time}")

        except requests.exceptions.RequestException as e:
            print(f"网络请求失败: {e}")
        except ValueError as e:
            print(f"API 返回结果异常: {e}")
        except Exception as e:
            print(f"未知错误: {e}")
```

对话类聊天V2

- 接口说明： 对话类聊天接口，和ChatQuery区别是，ChatQueryV2返回结构更加规范
  a. 流式返回：标准的SSE返回（区别于chat_query,每个消息体里没有嵌套data字段）；
  b. 非流式返回：返回完整的消息内容（json格式返回，区别于chat_query,返回不是SSE格式的）；
  c. 其他与ChatQuery完全一致

Post /chat_query_v2

请求 (app.ChatQueryRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧对话ID
Query	string	true	用户询问内容
ResponseMode	string	false	返回数据的格式，枚举值：streaming、blocking，默认：streaming

QueryExtends	app.QueryExtendsInfo	false	询问扩展信息
UserID	string	true	用户ID

响应 (app.MessageStreamResponse):

参数	类型	是否必填	说明
Data	string	false	EventStream(data: json)

	Postman示例	py代码示例	备注
请求示例:	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/chat_query' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "Query": "给我讲个故事", 6 "AppConversationID": "co6deaa1hp0kieia13bg",	代码块 1 def chat_query_v2(user_id: str, app_conversation_id: str, content: str): 2 """ 3 对话类聊天V2 4 :param user_id: 用户id 5 :param app_conversation_id: 对话id 6 :param content: 用户询问的内容 7 :return: 8 """ 9 url = base_url + "chat_query_v2" 10 headers = {'Apikey': api_key, 11 'Content-Type': 'application/json'} 12 data = {"UserID": user_id, 13 "AppConversationID": app_conversation_id, 14 "Query": content, 15 "ResponseMode": "blocking"} 16 try: 17 response = requests.post(url,	Files信息包括obs链接，关于如何将文件上传obs参考文档：Hi Agent Up服务接口调用说明 流式请求时： 请求头注意 `'Accept': 'text/event-stream'` 代码请求时：request设置 `stream=True`

```
    "ResponseMode": "streaming",
    "UserID": "321",
    "QueryExtends": {
        "Files": [
            {
                "Name": "图片.jpeg",
                "Path": "upload/full/59/e8/0304b770fd1dfebe806bf1ade61a4bc5ded18f969a1c8f803998eeb60573",
                "Size": 47525,
                "Url": "{{apiurl}}/api/proxy/down?Action=Download&Version=2022-01-
```

```
    headers=headers,
    data=json.dumps(data))

    response.raise_for_status()
    except requests.exceptions.RequestException as e:
        raise requests.exceptions.RequestException(f"网络请求失败: {e}")

    return response
```

	01&Path=upload%2Ffull%2F59%2Fe8%2F0304b770fd1dfebe806bf1ade61a4bc5ded18f969a1c8f803998eeb60573&TsAn		
返回示例：流式	代码块 1 { 2 event:text 3 data: {"event": "message_start", "task_id": "6b2f4720-a14e-413d-9783-024442a58e7d"} 4 5 event:text 6 data: {"event": "message", "task_id": "6b2f4720-a14e-413d-9783-024442a58e7d",	代码块 1 if __name__ == '__main__': 2 try: 3 uid = "321" 4 conversation_id = create_conversation(uid) 5 print(f"conversation_id: {conversation_id}") 6 7 question = "给我讲个故事" 8 resp = chat_query_v2(uid, conversation_id, question) 9 start_time = time.time() 10 task_id = None 11 12 for line in resp.iter_lines(chunk_size=4): 13 if line: # 过滤掉空行 14 line = line.decode("utf-8") 15 if line.startswith("data:"): 16 data = line.strip("data:").strip() 17 if data:	Data是标准的sse流式返回，用户需要自行解析data字段。字段含义如下： • event：事件类型，枚举值 ◦ message_start：消息开始 ◦ message：消息 ◦ message_end：消息结束 ◦ message_output_start：开始输出 ◦ message_output_end：输出结束 ◦ message_cost：消息消耗 ◦ suggestion：问题建议 ◦ suggestion_cost：问题建议消耗 ◦ agent_thought：思考 ◦ agent_thought_end：思考结束

```
    "id":
    "01JDHDFN
    H1XMJG89S
    EP2F49HDV
    ",
    "answer":
     "你",
    "created_
    at":
    173253013
    4,
    "conversa
    tion_id":

    "01JDHDFN
    GYRB130PZ
    SEDE8R70B
    "}
}
```

```
try:
 data = json.loads(data)
 if "event" in data and
data["event"] == "message":
     print(data["answer"],
end="")
 if "task_id" in data:
     task_id =
data["task_id"]
except
json.JSONDecodeError as e:
 print(f"JSON 解析失败:
{e}")
except KeyError as e:
 print(f"返回数据缺少必要字
段: {e}")
        print("\n")
        print(f"整体耗时:
{time.time() -
start_time}")
    except
requests.exceptions.Request
Exception as e:
        print(f"网络请求失
败: {e}")
    except ValueError as e:
        print(f"API 返回结果
异常: {e}")
    except Exception as e:
        print(f"未知错误:
{e}")
```

- message_replace：消息替换
- message_failed：消息失败
- knowledge_retrieve：知识库检索
- knowledge_retrieve_end：知识库检索结束
- agent_jump：agent跳转事件
- agent_take_over：agent接管最终回答的事件
- intention：意图
- qa_retrieve：问答库检索
- qa_retrieve_end：问答库检索结束
- knowledge_graph_retrieve：图谱库检索
- knowledge_graph_retrieve_end：图谱库检索结束
- terminology_retrieve：术语库检索
- terminology_retrieve_end：术语库检索结束
- tool_message：消息节点

    - think_message_output_start：思考开始
    - think_message：思考内容
    - think_message_output_end：思考结束
- task_id：任务ID，等效于当前对话的全局消息ID
- id：消息ID，包括当前对话的全局消息ID和嵌套的子消息ID：全局消息ID作为其他接口的MessageID入参；对话时如果调用工具等，会生成子消息ID。流式场景下，最外层的message_start、message_end里的ID为全局消息ID；非流式场景只会返回全局消息ID。
- answer：模型回复内容；
- created_at：消息创建时间；

			• conversation_id：模型会话ID，作用与app_conversation_id一致。区别为：app_conversation_id作为用户感知的一个会话的标识，具有业务属性；conversation_id只作为底层模型交互的会话的标识，屏蔽用户感知
返回示例： 非流式	代码块 1 { 2 "event": "message", 3 "task_id": "01JFW1QJJBRNVZ0B7JH0E034ST", 4 "id": "01JFW1QJJBRNVZ0B7JH0E034ST", 5 "conversation_id": "01JFW1P2DC7HBVK1383KCN494W", 6 "answer": "你好！你是想分享这	代码块 1 if __name__ == '__main__': 2 try: 3 uid = "321" 4 conversation_id = create_conversation(uid) 5 print(f"conversation_id: {conversation_id}") 6 7 question = "给我讲个故事" 8 resp = chat_query_v2(uid, conversation_id, question) 9 start_time = time.time() 10 11 try: 12 result = resp.json() 13 except json.JSONDecodeError as e: 14 raise ValueError(f"JSON 解析失败: {e}") 15 16 print("解析返回的 JSON 数据: ") 17 for key, value in result.items():	推荐使用chat_query_v2 返回参数： think_messages：推理思考消息列表。开启模型配置推理过程开关，则输出智能体对话中推理的过程消息，如果智能体中包含多个可推理模型对话，则返回每次推理的信息。 tool_messages：工具消息列表。工作流中消息节点输出的消息。

```
个图片文件的相关信息呢，还是有关于这个文件有什么问题想询问，比如如何打开、保存之类的？",
  "created_at": 1735034398,
  "think_messages": ["推理过程消息内容"],
  "tool_messages": ["工具消息输出内容"]
}
```

```
            print(f"{key}: {value}")

        print("\n")
        print(f"整体耗时: {time.time() - start_time}")

    except requests.exceptions.RequestException as e:
        print(f"网络请求失败: {e}")
    except ValueError as e:
        print(f"API 返回结果异常: {e}")
    except Exception as e:
        print(f"未知错误: {e}")
```

重新生成回复

- 接口说明：用于对话类聊天接口，重新生成回复（所有流式接口都是SSE）。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation接口生成；
  c. UserID：用户标识，用于定义终端用户的身份，方便检索、统计。 由开发者定义规则，需保证用户标识在应用内唯一。（长度：1-20）；
  d. MessageID：上轮对话返回的ID；
- response body参数说明：

a. Data：SSE格式的输出；

Post /query_again

请求 (app.QueryAgainRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧对话ID
MessageID	string	true	消息ID
UserID	string	true	用户ID

响应 (app.MessageStreamResponse):

参数	类型	是否必填	说明
Data	string	false	EventStream(data: json)

	Postman	python代码	备注
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/query_again' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppConversationID": "co6deaa1hp0kieia13bg", 6 "MessageID": "01HTH3FGMYMSYDH8J	代码块 1 def query_again(user_id: str, app_conversation_id: str, message_id: str): 2 url = base_url + "query_again" 3 headers = {'Apikey': api_key, 4 'Content-Type': 'application/json', 5 'Accept': 'text/event-stream'} 6 data = { 7 "AppConversationID": app_conversation_id,	只有流式

```
    YY3V7AGE9",
7       "UserID":
    "321"
8   }'
```

```
8           "MessageID":
    message_id,
9           "UserID":
    user_id
10      }
11
12      try:
13          response =
    requests.post(url,
    headers=headers,
    data=json.dumps(data),
    stream=True)
14
    response.raise_for_statu
    s()
15      except
    requests.exceptions.Requ
    estException as e:
16          raise
    requests.exceptions.Requ
    estException(f"网络请求失
    败: {e}")
17
18      return response
```

返回示例

代码块

```
1   {
2   event:text
3   data:data:
    {"event":
    "message_start",
    "task_id":
    "6b2f4720-a14e-
    413d-9783-
    024442a58e7d"}
4
5   event:text
6   data:data:
    {"event":
    "message",
    "task_id":
    "6b2f4720-a14e-
    413d-9783-
    024442a58e7d",
    "id":
```

代码块

```
1   if __name__ ==
    '__main__':
2       try:
3           uid = "321"
4
    app_conversation_id =
    "cv5c9rl93q03lmornb2g"
5           message_id =
    "01JNR12H305AET4ZZ7TFT08
    MAN"
6
7           resp =
    query_again(uid,
    app_conversation_id,
    message_id)
8           start_time =
    time.time()
9           task_id = None
10
```

```
    "01JDHDFNH1XMJG89SEP2F49HDV",
    "answer": "你",
    "created_at": 1732530134,
    "conversation_id": "01JDHDFNGYRB130PZSEDE8R70B"}
}
```

```python
        for line in resp.iter_lines(chunk_size=4):
            if line:  # 过滤掉空行
                line = line.decode("utf-8")
                if line.startswith("data:"):
                    data = line.strip("data:").strip()
                    if data:
                        try:
                            data = json.loads(data)
                            if "event" in data and data["event"] == "message":
                                print(data["answer"], end="")
                            if "task_id" in data:
                                task_id = data["task_id"]
                        except json.JSONDecodeError as e:
                            print(f"JSON 解析失败: {e}")
        except KeyError as e:
            print(f"返回数据缺少必要字段: {e}")
```

```
27
28          print("\n")
29          print(f"整体耗时: {time.time() - start_time}")
30
31      except requests.exceptions.RequestException as e:
32          print(f"网络请求失败: {e}")
33      except ValueError as e:
34          print(f"API 返回结果异常: {e}")
35      except Exception as e:
36          print(f"未知错误: {e}")
```

重新生成回复V2

- 接口说明： 用于对话类聊天接口，重新生成回复（所有流式接口都是SSE），和QueryAgain区别是，QueryAgainV2返回结构更加规范
  a. 流式返回：标准的SSE返回（区别于query_again,每个消息体里没有嵌套data字段）；
  b. 其他与QueryAgain完全一致
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation接口生成；
  c. UserID：用户标识，用于定义终端用户的身份，方便检索、统计。 由开发者定义规则，需保证用户标识在应用内唯一。（长度：1-20）；
  d. MessageID：上轮对话返回的ID；
- response body参数说明：
  a. Data：SSE格式的输出；

Post /query_again

请求 (app.QueryAgainRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧对话ID
MessageID	string	true	消息ID
UserID	string	true	用户ID

响应 (app.MessageStreamResponse):

参数	类型	是否必填	说明
Data	string	false	EventStream(data: json)

	Postman	python代码	备注
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/query_again_v2' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppConversationID": "co6deaa1hp0kieia13bg", 6 "MessageID": "01HTH3FGMYMSYDH8JYY3V7AGE9", 7 "UserID": "321" 8 }'	代码块 1 def query_again2(user_id: str, app_conversation_id: str, message_id: str): 2 url = base_url + "query_again" 3 headers = {'Apikey': api_key, 4 'Content-Type': 'application/json', 5 'Accept': 'text/event-stream'} 6 data = { 7 "AppConversationID": app_conversation_id, 8 "MessageID": message_id, 9 "UserID": user_id 10 } 11	只有流式

```
    try:
        response = requests.post(url, headers=headers, data=json.dumps(data), stream=True)

        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        raise requests.exceptions.RequestException(f"网络请求失败: {e}")

    return response
```

返回示例

代码块

```
{
event:text
data: {"event": "message_start", "task_id": "6b2f4720-a14e-413d-9783-024442a58e7d"}

event:text
data: {"event": "message", "task_id": "6b2f4720-a14e-413d-9783-024442a58e7d", "id": "01JDHDFNH1XMJG89SEP2F49HDV", "answer": "你", "created_at": 1732530134, "conversation_id":
```

代码块

```
if __name__ == '__main__':
    try:
        uid = "321"

        app_conversation_id = "cv5c9rl93q03lmornb2g"
        message_id = "01JNR12H305AET4ZZ7TFT08MAN"

        resp = query_again2(uid, app_conversation_id, message_id)
        start_time = time.time()
        task_id = None

        for line in resp.iter_lines(chunk_size=4):
            if line:  # 过滤掉空行
```

```
        "01JDHDFNGYRB130PZSEDE8R70B"}
    }
```

```
                line = line.decode("utf-8")
                if line.startswith("data:"):
                    data = line.strip("data:").strip()
                    if data:
                        try:
                            data = json.loads(data)
                            if "event" in data and data["event"] == "message":
                                print(data["answer"], end="")
                            if "task_id" in data:
                                task_id = data["task_id"]
                        except json.JSONDecodeError as e:
                            print(f"JSON 解析失败: {e}")
                        except KeyError as e:
                            print(f"返回数据缺少必要字段: {e}")
        print("\n")
        print(f"整体耗时: {time.time() - start_time}")
```

```
30
31      except requests.exceptions.RequestException as e:
32          print(f"网络请求失败: {e}")
33      except ValueError as e:
34          print(f"API 返回结果异常: {e}")
35      except Exception as e:
36          print(f"未知错误: {e}")
```

获取应用配置

- 接口说明： 获取应用配置。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. UserID：用户标识，用于定义终端用户的身份；

Post /get_app_config_preview

请求 (app.GetAppConfigPreviewRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID，此接口不需要强校验用户ID

响应 (app.GetAppConfigPreviewResponse):

参数	类型	是否必填	说明
VariableConfigs	list[app.VariableConfig]	false	变量配置
Name	string	false	名称

OpenMessage	string	false	开场白文本
OpenQuery	list[string]	false	开场白问题
Icon	string	false	图标
Background	string	false	图标背景色
MessageTimes	i32	false	生成更多类似
SuggestEnabled	bool	false	会话建议
BaseResp	base.BaseResp	false	

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/get_app_config_preview' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "UserID": "321" 6 }'	普通接口调用，暂不提供
返回示例	代码块 1 { 2 "Name": "dd", 3 "OpenMessage": "", 4 "OpenQuery": [], 5 "Icon": "MobilePhone", 6 "Background": "background_1", 7 "MessageTimes": 0, 8 "SuggestEnabled": false, 9 "Image": "",	普通接口调用，暂不提供

```
10        "AgentMode":
    "Single",
11
    "AgentNodeSuggestEnable
    dMap": {},
12        "VoiceConfig": {},
13        "BaseResp": null
14    }
```

获取对话列表

- 接口说明：获取对话列表，由CreateConversation接口生成的会话列表。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. UserID：用户标识，用于定义终端用户的身份；

Post /get_conversation_list

请求 (app.GetConversationListRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (app.GetConversationListResponse):

参数	类型	是否必填	说明
ConversationList	list[app.AppConversationBrief]	false	对话列表
BaseResp	base.BaseResp	false	

	Postman	python代码

请求示例	代码块 curl --location '{{apiurl}}/api/proxy/api/v1/get_conversation_list' \ --header 'Apikey: co6dd4i1hp0kieia137g' \ --header 'Content-Type: application/json' \ --data '{ "UserID": "321" }'	普通接口调用，暂不提供

返回示例	代码块
	{
	"ConversationList": [
	{
	"AppConversationID": "ctbat448v1t2jaap2kag",
	"ConversationName": "新的会话",
	"CreateTime": "2024-12-09 16:47:12",
	"LastChatTime": "2024-12-09 18:47:05",
	"EmptyConversation": false
	}
	],
	"BaseResp": null
	}
		普通接口调用，暂不提供

获取对话变量输入

- 接口说明：获取对话变量输入，变量在平台应用编排界面设置。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}。
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation接口生成；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /get_conversation_inputs

请求 (app.GetConversationInputsRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧会话ID
UserID	string	true	用户ID

响应 (app.GetConversationInputsResponse):

参数	类型	是否必填	说明
Inputs	map[string, string]	false	变量输入
BaseResp	base.BaseResp	false	

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/get_conversation_inputs' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{	普通接口调用，暂不提供

```
5       "AppKey":
    "co6dd4i1hp0kieia137g",
6       "UserID": "321"
7   }'
```

返回示例	代码块
	1 {
	2 "Inputs": {
	3 "var": "val"
	4 },
	5 "BaseResp": null
	6 }
		普通接口调用，暂不提供

更新会话

- 接口说明：更新会话，包括会话名称、变量。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation接口生成；
  c. ConversationName：会话名称；
  d. Inputs：变量输入；
  e. UserID：用户标识，用于定义终端用户的身份；

Post /update_conversation

请求 (app.UpdateConversationRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧会话ID

ConversationName	string	false	修改会话名称
Inputs	map[string, string]	false	修改变量输入
UserID	string	true	用户ID

响应 (common.EmptyResponse):

参数	类型	是否必填	说明

	Postman	python代码

请求示例	代码块
	1 curl --location '{{apiurl}}/api/proxy/api/v1/update_conversation' \
	2 --header 'Apikey: co6dd4i1hp0kieia137g' \
	3 --header 'Content-Type: application/json' \
	4 --data '{
	5 "AppConversationID": "co6deaa1hp0kieia13bg",
	6 "ConversationName": "更新会话",
	7 "Inputs": {
	8 "var": "variable"
	9 },
	10 "UserID": "321"
	11 }'
		普通接口调用，暂不提供

返回示例	代码块 1 {}	普通接口调用，暂不提供

删除会话

- 接口说明：删除会话。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}。
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation接口生成；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /delete_conversation

请求 (app.DeleteConversationRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧会话ID
UserID	string	true	用户ID

响应 (common.EmptyResponse):

参数	类型	是否必填	说明

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/delete_conversation' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{	普通接口调用，暂不提供

	5 "AppConversationID": "co6deaa1hp0kieia13bg", 6 "UserID": "321" 7 } 8 '	
返回示例	代码块 1 {}	普通接口调用，暂不提供

停止响应

- 接口说明：停止响应，在聊天类接口输出过程中，强制停止。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. TaskID：任务ID，由SendMessage(#SendMessage)接口生成；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /stop_message

请求 (app.StopMessageRequest):

参数	类型	是否必填	说明
TaskID	string	true	任务ID
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (common.EmptyResponse):

参数	类型	是否必填	说明

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/stop_message' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "co6dd4i1hp0kieia137g", 6 "TaskID": "be80e08f-bb46-46c5-a687-15824a7c4107", 7 "UserID": "321" 8 } 9 '	普通接口调用，暂不提供
返回示例	代码块 1 {}	普通接口调用，暂不提供

清除会话消息

- 接口说明：在聊天类智能体中，清楚指定会话下消息。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation 接口生成；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /clear_message

请求 (app.ClearMessageRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧会话ID
UserID	string	true	用户ID

响应 (common.EmptyResponse):

参数	类型	是否必填	说明

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/clear_message' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "co6dd4i1hp0kieia137g", 6 "AppConversationID": "ct4m7gmoc93a9ns2sngg", 7 "UserID": "321" 8 } 9 '	普通接口调用，暂不提供
返回示例	代码块 1 {}	普通接口调用，暂不提供

获取会话历史消息列表

- 接口说明：获取会话历史消息列表，对话轮数的消息。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}。
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppConversationID：会话ID，由CreateConversation 接口生成；
  c. Limit：限制数量；
  d. UserID：用户标识，用于定义终端用户的身份；

Post /get_conversation_messages

请求 (app.GetConversationMessagesRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
AppConversationID	string	true	应用侧会话ID
Limit	i32	true	限制数量
UserID	string	true	用户ID

响应 (app.GetConversationMessagesResponse):

参数	类型	是否必填	说明
Messages	list[app.ChatMessageInfo]	false	对话消息列表
BaseResp	base.BaseResp	false	

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/get_conversation_messages' \	普通接口调用，暂不提供

```
--header 'Apikey: co6dd4i1hp0kieia137g' \
--header 'Content-Type: application/json' \
--data '{
    "AppKey": "co6dd4i1hp0kieia137g",
    "Limit": 100,
    "AppConversationID": "co6deaa1hp0kieia13bg",
    "UserID": "321"
}'
```

返回示例

代码块


```
{
    "Messages": [
        {
            "ConversationID": "01JENGF8YA4NWT8AHP0WD9PTT6",
            "QueryID": "ctbcla1pgge5cea2drig",
            "Query": "北京天气",
            "AnswerInfo": {
                "Answer": "您想知道最近北京的天气情况吗？我可以帮您查询最新的天气信息并提供给您。\n\n根据今天的天气预报，北京的天气状况如下：\n- 日期：2023年11月28日\n- 温度：5°C到15°C\n- 降水概率：60%\n- 风速：3-4级\n\n请注意这些信息可能随时更新，保暖的同时出门记得携带雨具。您还可以通过网站、手机应用或者智能助手来查看更详细的天气信息。需要更具体的天气信息吗？比如特
```


普通接口调用，暂不提供

```
定时间的天气或者是未来几天的天气预报？",
    "MessageID": "01JENGF8YABMJBFG5YV1RB1J27",
    "CreatedTime": 1733741225,
    "TaskID": "01JENGF8YABMJBFG5YV1RB1J27",
                "Like": 0,
    "TotalTokens": 125,
    "Latency": 9.897,
    "TracingJsonStr": "",
    "RetrieverResource": false
            },
    "OtherAnswers": [],
            "Inputs": {}
        }
    ],
    "BaseResp": null
}
```


获取消息详情

- 接口说明：获取消息详情，包括消息的返回内容，消耗的token等。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；

b. MessageID：消息ID，由聊天接口返回；

c. UserID：用户标识，用于定义终端用户的身份；

Post /get_message_info

请求 (app.GetMessageInfoRequest):

参数	类型	是否必填	说明
MessageID	string	true	消息ID
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (app.GetMessageInfoResponse):

参数	类型	是否必填	说明
MessageInfo	app.ChatMessageInfo	false	消息详情
BaseResp	base.BaseResp	false	

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/get_message_info' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "co6dd4i1hp0kieia137g",	普通接口调用，暂不提供

```
6   "MessageID": "01HTH35SQHX8WGS006NTWVBSN7",
7       "UserID": "321"
8   }'
```

返回示例

代码块

```
1   {
2   "MessageInfo": {
3   "ConversationID": "01JENGF8YA4NWT8AHP0WD9PTT6",
4   "QueryID": "ctbcla1pgge5cea2drig",
5   "Query": "北京天气",
6   "AnswerInfo": {
7   "Answer": "您想知道最近北京的天气情况吗？我可以帮您查询最新的天气信息并提供给您。\n\n根据今天的天气预报，北京的天气状况如下：\n- 日期：2023年11月28日\n- 温度：5°C到15°C\n- 降水概率：60%\n- 风速：3-4级\n\n请注意这些信息可能随时更新，保暖的同时出门
```

这里作为提取『引用归属』的代码示例：（特别鸣谢 黄冬 ）

代码块

```python
def get_message_info(self, message_id):
        """
        根据消息 ID 获取消息的详细信息，特别是文档相关信息。
        参数:
        - message_id (str): 消息的 ID。
        返回:
        dict: 包含消息详细信息的字典，如果出现错误则返回 None。
        """
        url = base_url+"/api/proxy/api/v1/get_message_info"
        headers = {"Apikey": self.api_key, "Content-Type": "application/json"}
        data = {"MessageID": message_id, "UserID": self.user_id}
        try:
            response = requests.get(url, headers=headers, json=data)
            response.raise_for_status()

            result = response.json()['MessageInfo']['AnswerInfo']['TracingJsonStr']

            result = json.loads(result)
            # 遍历解析后的结果列表
            for item in result:
                # 找到知识检索结束的事件
```

```
记得携带雨具。您还可以通过网站、手机应用或者智能助手来查看更详细的天气信息。需要更具体的天气信息吗？比如特定时间的天气或者是未来几天的天气预报？",
    "MessageID": "01JENGF8YABMJBFG5YV1RB1J27",
    "CreatedTime": 1733741225,
    "TaskID": "01JENGF8YABMJBFG5YV1RB1J27",
    "Like": 0,
    "TotalTokens": 125,
    "Latency": 9.897,
    "TracingJsonStr": "",
    "RetrieverResource": false
        },
    "OtherAnswers": [],
    "Inputs": null
    },
    "BaseResp": null
}
```

```python
                if item["event"] == "knowledge_retrieve_end":
                    # 遍历该事件中文档输出列表
                    for doc in item["docs"]["outputList"]:
                        # 提取文档的名称
                        document_name = doc["metadata"]["document_name"]
                        # 提取文档的 OBS 链接地址
                        document_obs_url = doc["metadata"]["document_obs_url"]
                        # 打印文档名称
                        print(f"文档名称: {document_name}")
                        s3_url = f"https://{self.host}/api/proxy/down?Action=Download&Version=2022-01-01&Path={document_obs_url}&IsAnonymous=true"
                        # 打印文档的 OBS 链接地址
                        print(f"OBS 链接地址: {s3_url}")
                        # 打印分隔线，用于区分不同文档信息
                        print("-" * 50)
            # 返回解析后的结果
            return result
        except requests.RequestException as e:
            # 处理请求过程中出现的异常，打印错误信息
            print(f"Request error: {e}")
        except (KeyError, json.JSONDecodeError) as e:
            # 处理 JSON 解析或键值提取过程中出现的异常，打印错误信息
            print(f"Error parsing response: {e}")
        # 如果出现异常，返回 None
        return None
```

删除消息

- 接口说明：删除消息，用户只能删除自己对话的消息，删除后，消息不会出现在对话上下文
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. MessageID：消息ID，由聊天接口返回；
  c. QueryID：询问ID，由get_message_info返回（重新生成时，通过query_id来归纳一个消息组）；
  d. UserID：用户标识，用于定义终端用户的身份；

Post /delete_message

请求 (app.DeleteMessageRequest):

参数	类型	是否必填	说明
MessageID	string	false	消息ID,传消息ID不能同时传询问ID
QueryID	string	false	询问ID,传询问ID不能同时传消息ID
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (common.EmptyResponse):

参数	类型	是否必填	说明

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/delete_message' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \	普通接口调用，暂不提供

	3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "co6dd4i1hp0kieia137g", 6 "MessageID": "01HTH35SQHX8WGS006NTWVBSN7", 7 "QueryID": "ed6d9dje447dhtf4a74hds", 8 "UserID": "321" 9 }'	
返回示例	代码块 1 {}	普通接口调用，没有提供的价值，如不会写可复制postman请求让大模型写

回答反馈评价（赞或踩）

- 接口说明：
  a. 聊天返回结果反馈评价（赞或踩）。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }};
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. MessageID：消息ID，由聊天接口返回；
  c. LikeType：反馈类型；
  d. UserID：用户标识，用于定义终端用户的身份；

Post /feedback

请求 (app.FeedbackRequest):

参数	类型	是否必填	说明

MessageID	string	true	消息ID
LikeType	common.LikeType	true	反馈类型
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (common.EmptyResponse):

参数	类型	是否必填	说明

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/feedback' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "co6dd4i1hp0kieia137g", 6 "MessageID": "01HTH35SQHX8WGS006NTWVBSN7", 7 "LikeType": 1, 8 "UserID": "321" 9 }'	普通接口调用，暂不提供
返回示例	代码块 1 {}	普通接口调用，暂不提供

多组回答时，设置某个回答为默认使用的回答

- 接口说明： 多组回答时，设置某个回答为默认使用的回答。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. MessageID：消息ID，由聊天接口返回；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /set_message_answer_used

请求 (app.SetMessageAnswerUsedRequest):

参数	类型	是否必填	说明
MessageID	string	true	消息ID
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (common.EmptyResponse):

参数	类型	是否必填	说明

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/set_message_answer_used' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "co6dd4i1hp0kieia137g",	普通接口调用，暂不提供

```
6       "MessageID":
    "01HTH35SQHX8WGS006NTWV
    BSN7",
7       "UserID": "321"
8   }'
```

返回示例	代码块 1 {}	普通接口调用，暂不提供

获取提问建议

- 接口说明：聊天类接口专用，用于获取提问建议。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. MessageID：消息ID，由聊天接口返回；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /get_suggested_questions

请求 (app.GetSuggestedQuestionsRequest):

参数	类型	是否必填	说明
MessageID	string	true	消息ID
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (app.GetSuggestedQuestionsResponse):

参数	类型	是否必填	说明

SuggestedQuestions	list[string]	false	推荐列表
BaseResp	base.BaseResp	false	

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/get_suggested_questions' \ 2 --header 'Apikey: co6dd4i1hp0kieia137g' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "co6dd4i1hp0kieia137g", 6 "MessageID": "01HTH35SQHX8WGS006NTWVBSN7", 7 "UserID": "321" 8 }'	普通接口调用，暂不提供
返回示例	代码块 1 { 2 "SuggestedQuestions": [3 "明天天气如何？", 4 "未来一周天气？", 5 "现在温度多少？" 6], 7 "BaseResp": null 8 }	普通接口调用，暂不提供

测试运行工作流(异步)

- 接口说明：应用类型为流程编排型时，测试运行工作流(异步)。

- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. InputData：输入json参数；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /run_app_workflow

请求 (app.RunAppWorkflowRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
InputData	string	true	输入json参数
UserID	string	true	用户ID

响应 (workflow.DebugWorkflowResponse):

参数	类型	是否必填	说明
runId	string	false	

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/run_app_workflow' \ 2 --header 'Apikey: cnndp5ubmcq0s7g3s8og' \ 3 --header 'Content-Type: application/json' \ 4 --data '{ 5 "AppKey": "cnndp5ubmcq0s7g3s8og", 6 "InputData": "{\"content\":\"我想上天\"}",	代码块 1 def run_workflow(text: str, max_retries: int = 3) -> Any | None: 2 """ 3 执行工作流并返回 runId 4 5 参数: 6 text (str): 输入文本内容 7 max_retries (int): 最大重试次数，默认3次 8 9 返回:

```
    "UserID": "321"
}'
```

```
        str: 成功返回 runId，失败返回 None
    """
    # 验证输入数据
    if not text.strip():
        raise ValueError("输入文本不能为空")

    API_URL = base_url+"run_app_workflow"
    HEADERS = {
        "Content-Type": "application/json",
        "Apikey": API_KEY
    }
    BASE_DATA = {
        "AppID": APP_ID,
        "UserID": USER_ID,
        "InputData": json.dumps({"input": text})
    }

    result = make_request(API_URL, HEADERS, BASE_DATA, max_retries)

    if result and 'runId' in result:
        return result['runId']
    return None
```

返回示例

代码块

```
{
    "runId": "c4213b25cd614424ade0009ceb445f36"
}
```

测试运行工作流(同步)

- 接口说明：应用类型为流程编排型时，测试运行工作流(同步)。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. InputData：输入json参数；
  c. UserID：用户标识，用于定义终端用户的身份；

Post /sync_run_app_workflow

请求 (app.SyncRunAppWorkflowRequest):

参数	类型	是否必填	说明
AppKey	string	false	应用key,已废弃可不传
InputData	string	true	输入json参数
UserID	string	true	用户ID

响应 (workflow.DebugWorkflowResponse):

参数	类型	是否必填	说明
runId	string	false	

	Postman	python代码
请求示例	代码块 1 curl --location '{{apiurl}}/api/proxy/api/v1/sync_run_app_workflow' \ 2 --header 'Apikey: cnndp5ubmcq0s7g3s8og' \ 3 --header 'Content-Type: application/json' \ 4 --data '{	代码块 1 def run_workflow(text: str, max_retries: int = 3) -> Any | None: 2 """ 3 执行工作流并返回 runId 4 5 参数: 6 text (str): 输入文本内容

```
    "AppKey": "cnndp5ubmcq0s7g3s8og",
    "InputData": "{\"content\":\"我想上天\"}",
    "UserID": "321"
}'
```

```
        max_retries (int): 最大重试次数，默认3次

    返回:
        str: 成功返回 runId，失败返回 None
    """
    # 验证输入数据
    if not text.strip():
        raise ValueError("输入文本不能为空")

    API_URL = base_url+"sync_run_app_workflow"
    HEADERS = {
        "Content-Type": "application/json",
        "Apikey": API_KEY
    }
    BASE_DATA = {
        "AppID": APP_ID,
        "UserID": USER_ID,
        "InputData": json.dumps({"input": text})
    }

    result = make_request(API_URL, HEADERS, BASE_DATA, max_retries)

    if result and 'runId' in result:
        return result['runId']
    return None
```

返回示例

代码块

```
{
    "status": "success",
    "nodes": {
```

```
    "ctbe233ml3fgfs3u2skg": {
            "input": "{\"dd\":\"你好\"}",
            "output": "{\"dd\":\"你好\"}",
            "status": "success",
            "message": "",
            "costMs": 3,
            "costToken": 0,
            "nodeType": "start"
        },
        "ctbe233ml3fgfs3u2sl0": {
            "input": "{\"output\":\"你好\"}",
            "output": "{\"output\":\"你好\"}",
            "status": "success",
            "message": "",
            "costMs": 8,
            "costToken": 0,
            "nodeType": "end"
        }
    },
    "steps": null,
    "costMs": 16
}
```

查询测试运行工作流运行进展(异步)

- 接口说明：应用类型为流程编排型时，查询测试运行工作流运行进展(异步)。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：
  a. AppKey：赋值为{{ ApiKey }}（值同header的Apikey）；
  b. AppID：应用ID；
  c. RunID：运行ID；

Post /query_run_app_process

请求 (app.QueryRunAppProcessRequest):

参数	类型	是否必填	说明
AppID	string	true	
RunID	string	true	
AppKey	string	false	应用key,已废弃可不传
UserID	string	true	用户ID

响应 (workflow.DebugWorkflowResponse):

参数	类型	是否必填	说明
runId	string	false	
status	string	false	
nodes	map[string, workflow.Node]	false	
steps	list[string]	false	
code	i32	false	
message	string	false	

	Postman	python代码

请求示例

代码块

```
curl --location '{{apiurl}}/api/proxy/api/v1/query_run_app_process' \
--header 'Apikey: cnndp5ubmcq0s7g3s8og' \
--header 'Content-Type: application/json' \
--data '{
    "AppID": "cno0vo5j8i35udgfhkh0",
    "RunID": "4deecd7c453a46febfe6d84b179e2ed9",
    "UserID": "321",
    "AppKey": "cnndp5ubmcq0s7g3s8og"
}'
```

轮询结果，特别鸣谢 黄冬

代码块

```python
def get_process(run_id, max_retries=10):

    query_y_run_app_process_url = base_url+"query_run_app_process"
    data = {
        "AppID": APP_ID,
        "UserID": USER_ID,
        "RunID": run_id
    }
    headers = {"Content-Type": "application/json", "Apikey": API_KEY}

    retry_count = 0
    while True:
        result = make_request(query_y_run_app_process_url, headers, data, 1)
        if result:
            if result.get('status') == "success":
                print(result)
                return result
            print(result)

        # 重试逻辑
        retry_count += 1
        if retry_count >= max_retries:
            print("Max retries exceeded")
            return None

        # 指数退避等待（2, 4, 8, 16...秒）
        time.sleep(min(2 ** retry_count, 30))  # 最大间隔30秒
```

返回示例

代码块

```
{
    "status": "success",
    "nodes": {
        "ctbe233ml3fgfs3u2skg": {
            "input": "{\"dd\":\"你好\"}",
            "output": "{\"dd\":\"你好\"}",
            "status": "success",
            "message": "",
            "costMs": 3,
            "costToken": 0,
            "nodeType": "start"
        },
        "ctbe233ml3fgfs3u2sl0": {
            "input": "{\"output\":\"你好\"}",
            "output": "{\"output\":\"你好\"}",
            "status": "success",
            "message": "",
            "costMs": 8,
            "costToken": 0,
            "nodeType": "end"
        }
    },
    "steps": null,
```

```
    "costMs": 16
}
```

查询用户oauth2登录信息的接口

- 接口说明：查询用户oauth2登录信息的接口。
- 请求header说明：
  a. 请求header需要加上Apikey={{ ApiKey }}；
- body参数说明：

Post /list_oauth2_token

请求 (common.EmptyRequest):

参数	类型	是否必填	说明
UserID	string	true	用户ID

响应 (plugin.ListOauth2TokenResp):

参数	类型	是否必填	说明
Total	i64	true	
Items	list[plugin.Oauth2TokenItem]	true	

请求示例：

```
curl --location '{{apiurl}}/api/proxy/api/v1/list_oauth2_token' \
--header 'Apikey: cnndp5ubmcq0s7g3s8og' \
--header 'Content-Type: application/json' \
--data '{
    "UserID": "321"
}'
```

触发事件触发器

- 接口说明：触发发布后的智能体中事件触发器执行工作。
- 请求header说明：
  a. 请求header需要加上Authorization=Bearer {{ WebHookBearerToken }}；WebHookBearToken 为事件触发器中配置的 bearer token.
- body参数说明：无固定（json）。根据具体的事件触发器中设定的请求参数。

Post /trigger/webhook?key={{WebHookKey}}

- query 参数说明:
  - WebHookKey: 智能体中配置的事件触发器分配的唯一标识。

请求 (common.EmptyRequest):

参数	类型	是否必填	说明
无固定			**根据具体的事件触发器中设定的请求参数

响应 (plugin.EmptyResponse):

参数	类型	是否必填	说明

请求示例：

代码块

```
curl --location '{{apiurl}}/api/proxy/api/v1/trigger/webhook?key={{WebHookKey}}' \
--header 'Authorization: Bearer {{WebHookBearerToken}}' \
--header 'Content-Type: application/json' \
--data '{
    "a": "hnf"
}'

```

结构体定义

app.AppConversationBrief

说明:

参数	类型	是否必填	说明
AppConversationID	string	false	应用侧会话ID
ConversationName	string	false	会话名称

app.ChatMessageInfo

说明:

参数	类型	是否必填	说明
ConversationID	string	false	会话ID
QueryID	string	false	询问ID
Query	string	false	询问内容
AnswerInfo	app.MessageAnswerInfo	false	回答信息
OtherAnswers	list[app.MessageAnswerInfo]	false	其他回答
QueryExtends	app.QueryExtendsInfo	false	询问扩展信息

app.MessageAnswerInfo

说明:

参数	类型	是否必填	说明
Answer	string	false	答复内容
MessageID	string	false	消息ID
CreatedTime	i32	false	时间戳
TaskID	string	false	任务ID
Like	common.LikeType	false	点赞
TotalTokens	i32	false	token消耗

Latency	double	false	耗时, 单位秒
TracingJsonStr	string	false	tracing, debug相关接口才返回

app.FileInfo

说明:

参数	类型	是否必填	说明
Path	string	false	路径
Name	string	false	名称
Size	i64	false	大小
Url	string	false	链接

app.QueryExtendsInfo

说明:

参数	类型	是否必填	说明
Files	list[app.FileInfo]	false	文件列表

app.VariableConfig

说明:

参数	类型	是否必填	说明
Key	string	true	变量KEY
Name	string	true	字段名称
Description	string	false	描述
Required	bool	true	是否必填
VariableType	common.VariableType	true	变量类型：文本Text 枚举Enum 段落Paragraph
EnumValues	list[string]	false	

			枚举值, 变量类型是Enum时必填
TextMaxLength	i32	false	文本最大长度限制, 变量类型是Text时必填

base.BaseResp

说明:

参数	类型	是否必填	说明
StatusMessage	string	false	
StatusCode	i32	false	
Extra	map[string, string]	false	

common.LikeType

说明: 枚举类型

参数	类型	是否必填	说明
Dislike	i32	false	枚举值:-1
Default	i32	false	枚举值:0
Like	i32	false	枚举值:1

common.ListOption

说明: ListOption

参数	类型	是否必填	说明
Sort	list[common.Sorter]	false	
PageNumber	i32	false	排序字段
PageSize	i32	false	页码

common.Sorter

说明: Sorter

参数	类型	是否必填	说明
SortField	string	true	排序字段，填snake_case，例如操作时间就填created_at，更新时间填updated_at
SortOrder	string	false	排序顺序，默认desc，可填desc, asc

common.VariableType

说明: 变量类型

参数	类型	是否必填	说明

workflow.DebugInputVariable

说明:

参数	类型	是否必填	说明
Name	string	true	变量名称
StringValue	string	false	string 类型的变量值，StringValue, IntValue, DoubleValue, BoolValue 四个字段只能选一个
IntValue	i64	false	int 类型的变量值，StringValue, IntValue, DoubleValue, BoolValue 四个字段只能选一个
DoubleValue	double	false	double 类型的变量值，StringValue, IntValue, DoubleValue, BoolValue 四个字段只能选一个
BoolValue	bool	false	bool 类型的变量值，StringValue, IntValue, DoubleValue, BoolValue 四个字段只能选一个

workflow.Node

说明:

参数	类型	是否必填	说明
input	string	false	
output	string	false	
status	string	false	
message	string	false	
costMs	i32	false	
costToken	i32	false	

plugin.Oauth2TokenItem

说明:

参数	类型	是否必填	说明
PluginID	string	true	插件ID
APPID	string	true	应用ID
APPUserID	string	true	应用用户ID
TokenExpiresAt	string	false	token 过期时间
IsTokenValid	bool	true	token 是否有效, 如果 插件认证配置更新, 未登录, token 过期 会标记失效
IsRefreshTokenValid	bool	true	refreshToken 是否有效, 如果 插件认证配置更新, 未登录, 无refrshToken 会标记失效